LFX Mentorship - Term 3 2025

KubeSlice: Implement Dynamic IPAM for the Slice Overlay Network

Candidate Information:

1. **Name** : Sai Sanjay Kottakota
2. **Github** : sanjay7178
3. **Email** : saisanjay7660@gmail.com
4. **Twitter** : sanjay7178
5. **Slack Username** :
6. **University Course**: Integrated Master's in Computer Science and Engineering (Current)
7. **University**: VIT University
8. **Time Zone**: IST (Indian Standard Time)

How did you find out about our mentorship program?

I found out about the LFX mentorship program through one of my friends. On exploring the site,I saw the program listed under open applications.

Why are you interested in this program?

Upon reviewing the project description and the associated repository issue, I recognized the critical importance of this feature for the community and users, and how its implementation would significantly enhance Kubeslice. The project necessitates resolving intricate challenges related to IP space optimization, state synchronization across clusters, and conflict-free subnet allocation. This ambitious undertaking offers invaluable learning opportunities in Kubernetes internals and multi-K8s cluster orchestration. This experience has solidified my conviction that cloud-native architectures represent the future of computing. Witnessing how an effective solution can directly influence operational efficiency and cost optimization inspires me to contribute to projects that address similar large-scale infrastructure issues.

What experience and knowledge/skills do you have that are applicable to the program ?

I have been active in Open Source contributions for more than a year now .I was selected for an Open Source fellowship, specifically Google Summer of Code 2025, with the

OpenScience Labs organization.I gained experience working with Docker Swarm and Podman, and developed a Docker plugin called swarm-external-secrets using Golang. . My participation in Hacktoberfest 2024 was instrumental in fostering my initial engagement with the open-source community.
I developed Kubebrowse, an academic open-source project focused on Kubernetes. This platform provides secure browser isolation with an integrated safe file viewer and phishing mitigation. Through this project, I gained in-depth knowledge of Kubernetes and Golang, including Kubernetes CRDs Controllers Operators , RBAC policies, orchestration of headless services , Istio service mesh , Falco, OTEL tracing and log monitoring with Fluentd. I also delved into the guacd protocol and efficient handling of distributed websockets using Go routines.

I've also been a significant contributor to Kubeslice, and my contribution previously on Kubetail, a CNCF project, has deepened my understanding of Kubernetes.

My Involvement in Kubeslice Project:

- **Slack:** Actively suggested and implemented improvements (username: Sanjay).
- **Examples Repository:**
  - **Bookinfo with mTLS (PR #46):** Implemented Istio Bookinfo with mTLS across two worker clusters, created secure multi-cluster configuration, and verified deployment on GKE via KubeSlice.
  - **Contour Ingress Gateway (PR #45):** Added production-ready Contour/Envoy ingress for Bookinfo, configured inter-cluster communication via OpenVPN, and provided detailed documentation.
- **CLI Improvements:**
  - **Cluster Health Monitoring (PR #90):** Developed `show-health` command for KubeSlice cluster status, enhancing observability.
- **Infrastructure Improvements:**
  - **Multi-Architecture Support (PRs #24, #8):** Added multi-architecture Docker build support for `cmd-nsmgr` and `cmd-exclude-prefixes-k8s`, improving cross-platform compatibility and implementing automated workflows.
- **Kubeslice cli Installation Bug (Issue #97)**
  - Identified critical installation failure with kubeslice-cli install --profile=minimal-demo on Kind
  - Documented worker node availability issues in kubeslice-system namespace
  - Discovered compatibility problems with GKE Autopilot clusters that lack default worker nodes
  - Provided detailed reproduction steps and logs to facilitate resolution

What do you hope to get out of this mentorship experience?

I am keen to contribute to this project within a large organization, as it will significantly advance my DevOps career. My enthusiasm for open-source development drives me to make meaningful contributions that benefit a broad audience. I seek to gain valuable insights and mentorship from the maintainers. By the end of this mentorship, I aspire to be more adept in Kubernetes and Go, and to possess a deeper understanding of Kubeslice and its components. Ultimately, the most significant outcome I hope for from this mentorship is the guidance and experience that will benefit me throughout my professional life.

Project Proposal

Title : Implement Dynamic IPAM for the Slice Overlay Network (Issue : #252)
Description :

🔖 Feature description
In the current KubeSlice design, IP address management (IPAM) for slice overlay networks is static and inefficient. A predefined CIDR block (e.g., 10.1.0.0/16) is divided into a fixed number of subnets regardless of how many clusters participate in the slice, leading to significant IP space wastage.

🎤 Pitch
This issue aims to implement a dynamic IPAM system that allocates IP subnets to clusters on demand and reclaims unused ranges when clusters leave the slice. It will ensure efficient address utilization, synchronization across clusters, and integration with the KubeSlice control plane.

✌️ Solution
A dynamic IPAM allocator integrated with the KubeSlice controller or sidecar component.
Support for on-demand IP allocation and subnet reclamation when clusters join or leave a slice.
Conflict resolution and state synchronization across clusters using CRDs or distributed storage.
Documentation on how the system works, configuration options, and edge case behaviour.

Feature Overview

I'm excited to propose my implementation approach for the Custom Topology Definition feature . This feature will transform KubeSlice's networking capabilities by moving beyond the current full-mesh topology to support flexible connection patterns like hub-spoke and partial mesh, significantly improving resource efficiency and network design flexibility.

Implementation Approach:

Core API Extension

I propose extending the SliceConfig CRD with a new `topologyConfig` field:

```go
// SliceConfigSpec extension
type SliceConfigSpec struct {
    // Existing fields...

    // TopologyConfig defines custom connectivity topology for the slice
    TopologyConfig *TopologyConfig `json:"topologyConfig,omitempty"`
}

// New topology configuration types
type TopologyConfig struct {
    // TopologyType defines the connectivity pattern
    TopologyType TopologyType `json:"topologyType,omitempty"`
    // CustomConnections defines explicit connections
    CustomConnections []ClusterConnection
`json:"customConnections,omitempty"`
    // HubCluster defines the hub cluster name for hub-spoke topology
    HubCluster string `json:"hubCluster,omitempty"`
    // ClusterVPNRoles defines VPN roles for each cluster
    ClusterVPNRoles []ClusterVPNRole `json:"clusterVpnRoles,omitempty"`
}

// Topology type enum
type TopologyType string

const (
    FULL_MESH    TopologyType = "full-mesh"
    PARTIAL_MESH TopologyType = "partial-mesh"
    HUB_SPOKE    TopologyType = "hub-spoke"
    CUSTOM       TopologyType = "custom"
)
```

Architecture Visualization

Connection Matrix Algorithm

The core of my implementation is a flexible connection matrix builder that generates the appropriate connections based on topology type:

```
func buildConnectionMap(clusterNames []string, topologyConfig
*TopologyConfig) map[Connection]struct{} {
    // Default to full mesh if no topology config is provided
    if topologyConfig == nil {
        return buildFullMeshConnections(clusterNames)
    }

    switch topologyConfig.TopologyType {
    case FULL_MESH, "":
        return buildFullMeshConnections(clusterNames)
    case HUB_SPOKE:
        return buildHubSpokeConnections(clusterNames,
topologyConfig.HubCluster)
    case PARTIAL_MESH, CUSTOM:
        return buildCustomConnections(topologyConfig.CustomConnections)
    default:
        return buildFullMeshConnections(clusterNames)
    }
}

func buildHubSpokeConnections(clusterNames []string, hubCluster string)
map[Connection]struct{} {
    connectionMap := make(map[Connection]struct{})

    // Validate hub cluster exists
    hubExists := false
    for _, cluster := range clusterNames {
        if cluster == hubCluster {
            hubExists = true
            break
        }
    }

    // Connect hub to all spokes
    if hubExists {
        for _, cluster := range clusterNames {
```

```
            if cluster != hubCluster {
                connectionMap[Connection{
                    source: hubCluster,
                    destination: cluster,
                }] = struct{}{}
            }
        }
    }

    return connectionMap
}
```

My implementation ensures backward compatibility by defaulting to full-mesh behavior when no topology configuration is provided. This allows for gradual adoption without disrupting existing deployments.

Changes in SliceConfig on how users will use

```
apiVersion: controller.kubeslice.io/v1alpha1
kind: SliceConfig
metadata:
  name: hub-spoke-slice
  namespace: kubeslice-controller
spec:
  clusters: ["hub-cluster", "spoke-1", "spoke-2", "spoke-3"]
  topologyConfig:
    topologyType: hub-spoke
    hubCluster: hub-cluster
    clusterVpnRoles:
      - clusterName: hub-cluster
        role: server
      - clusterName: spoke-1
        role: client
```

12-Week Implementation Plan

12-Week Implementation Plan for Dynamic IPAM

Week 1-2 (Sep 8 - Sep 21, 2025)

- Design and implement IPAMAllocation CRD with validation
- Develop core subnet allocation algorithm
- Set up testing infrastructure and initial unit tests

Week 3-4 (Sep 22 - Oct 5, 2025)

- Implement DynamicIPAMService with allocation/deallocation methods
- Develop reconciliation logic for state synchronization
- Create comprehensive test suite for core functionality

Week 5-6 (Oct 6 - Oct 19, 2025)

- Enhance SliceConfigService to support dynamic IPAM
- Update WorkerSliceConfigService for dynamic subnet allocation
- Implement WorkerSliceGatewayService integration

Week 7-8 (Oct 20 - Nov 2, 2025)

- Develop end-to-end testing in multi-cluster environment
- Test edge cases and concurrent operations
- Validate backward compatibility with existing deployments

Week 9-10 (Nov 3 - Nov 16, 2025)

- Optimize performance under load conditions
- Implement caching and batch operations
- Create user documentation and migration guides

Week 11-12 (Nov 17 - Nov 30, 2025)

- Conduct final testing across multiple environments
- Address feedback and implement final refinements
- Complete documentation and prepare for production deployment

Closing

I'm excited about the opportunity to implement this feature that will significantly enhance KubeSlice's scalability and flexibility. My background in distributed systems and networking makes me well-suited to tackle this project.

Thank you for considering my proposal.